

Лабораторная работа №12. Жизненный цикл Activity (Activity Lifecycle)

Цель работы: изучить жизненный цикл Activity в Android и научиться правильно обрабатывать переходы между состояниями. В результате лабораторной работы будет создано интерактивное приложение "**Grade Clicker**" (Кликер оценок), которое демонстрирует работу **lifecycle callbacks** и сохранение состояния при изменении конфигурации устройства.

Практическая часть

Описание приложения

Мы создадим приложение "**Grade Clicker**" — интерактивный кликер для набора баллов по курсу "Разработка мобильных приложений".

Механика:

- Пользователь кликает по изображению, набирая баллы
- За каждый клик начисляется **+5 баллов**
- При достижении определённого количества баллов изменяется: Отображаемая оценка; Фоновое изображение; Цвет интерфейса

Система оценок:

Баллы	Оценка	Сообщение
0-49	Неудовлетворительно	"Продолжайте работать!"
50-69	Удовлетворительно	"Вы получили зачёт!"
70-89	Хорошо	"Отличная работа!"
90-100	Отлично	"Вы успешно сдали курс!"

Шаг 1. Создание проекта

1. Откройте **Android Studio**
2. Выберите **File** → **New** → **New Project**
3. Выберите **Empty Activity** → **Next**
4. Укажите параметры:
 - **Name:** **Grade_Clicker**
 - **Package name:** **com.baume_фюо.gradeclicker**
 - **Minimum SDK:** API 25

5. Нажмите **Finish**

Дождитесь загрузки проекта.

Создание Git-репозитория

Перед началом работы создайте Git-репозиторий для отслеживания изменений и защиты от потери данных.

Вариант 1: Через Android Studio (рекомендуется)

1. В Android Studio выберите **VCS → Enable Version Control Integration**
2. Выберите **Git → OK**
3. Выберите **Git → Commit** (или **Ctrl+K**)
4. **Выберите все файлы**, введите сообщение: **"Initial commit"**
5. Нажмите **Commit**

Вариант 2: Через терминал

1. Откройте терминал в Android Studio (**View → Tool Windows → Terminal**)
2. Выполните команды:

git init

git add .

git commit -m "Initial commit"

Создание удалённого репозитория на GitHub

1. Откройте **GitHub** и войдите в аккаунт.
2. Нажмите **New repository** (зелёная кнопка)
3. **Repository name:** **Grade_Clicker_FIO** (где FIO — ваша фамилия)
4. Выберите **Public** (публичный репозиторий)
5. **НЕ** ставьте галочки на "Add README" и ".gitignore"
6. Нажмите **Create repository**
7. Скопируйте URL репозитория

Привязка локального репозитория к удалённому

В терминале Android Studio выполните:

git branch -M main

git remote add origin `<URL_репозитория>`

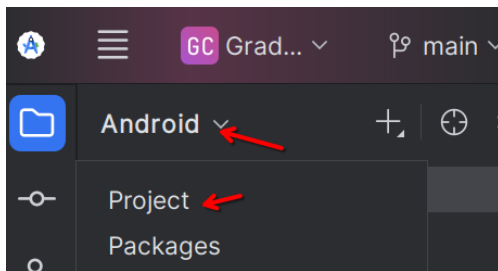
git push -u origin main

** Замените `<URL_репозитория>` на скопированный URL*

При первом **push** вас попросят ввести логин и пароль **GitHub**.

3. Создание папки **img**

1. В Android Studio переключитесь в режим просмотра **Project** (в левом верхнем углу панели Project)



2. Найдите **app**

3. Кликните правой кнопкой на папке **app**

4. Выберите **New** → **Directory**

5. Введите имя: **img**

6. Нажмите **OK**

Полный путь к папке: `/app/img/`

Шаг 2. Подготовка ресурсов

2.1. Создание изображений баллов в Krita

Мы создадим **5 изображений** для разных этапов набора баллов.

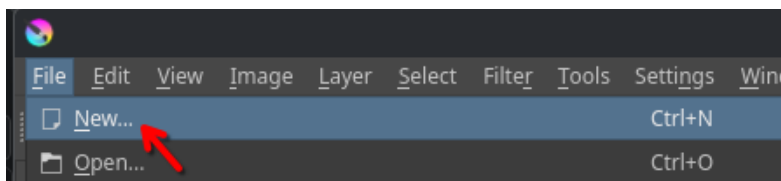
В данной лабораторной работе в качестве примера используется бесплатный редактор **Krita**, но вы можете использовать другие аналоги — Canva, Figma, Paint и др.

Создание изображений

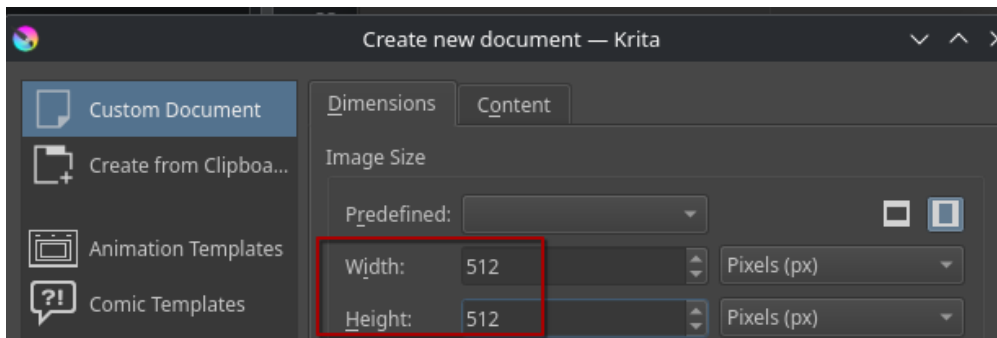
Создайте **5 файлов с баллами**:

1. **grade_0.png** (0-49 баллов):

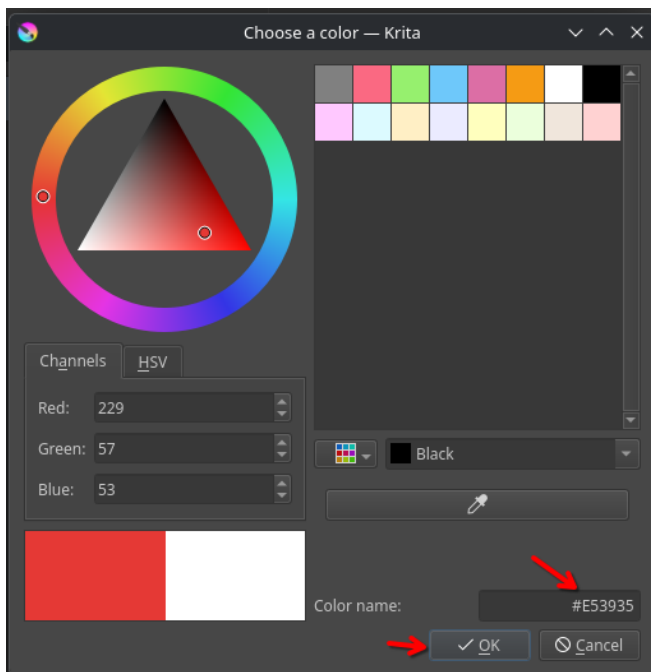
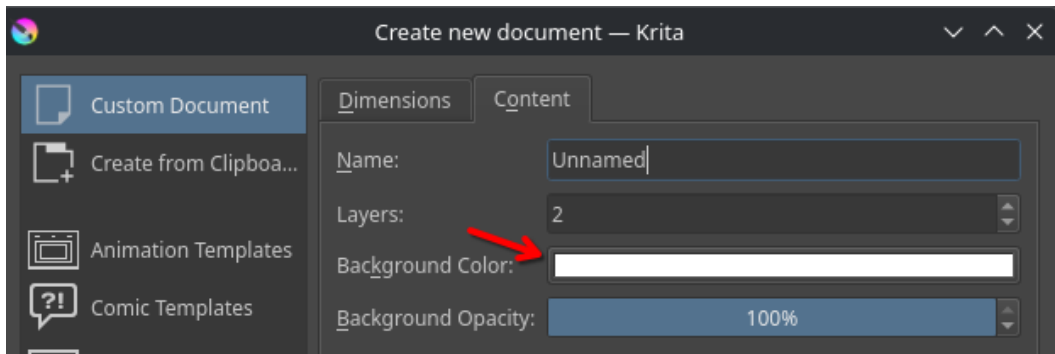
1. Откройте Krita → **File** → **New**



2. Размер: **512x512px**

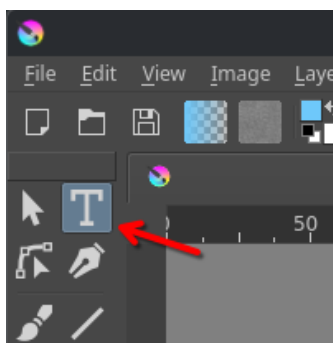


3. Фон: Красный (#E53935)



4. Create

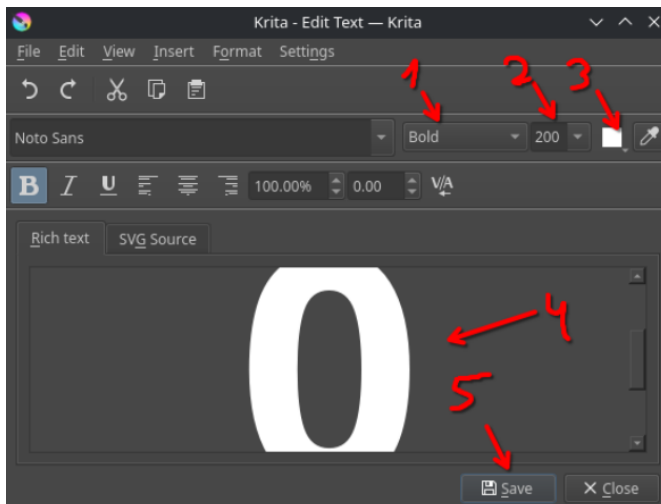
5. Инструмент **Text Tool (T)**



6. Напишите крупно: "0"

7. Цвет текста: **Белый**

8. Шрифт: **Жирный**, размер ~200



9. **File** → **Export** → **PNG**

10. Сохраните как **grade_0.png**

2. **grade_50.png** (50-69 баллов):

- Фон: Оранжевый (#FF9800)
- Текст: "50"
- Сохраните как **grade_50.png**

3. **grade_70.png** (70-89 баллов):

- Фон: Жёлтый (#FDD835)
- Текст: "70"
- Сохраните как **grade_70.png**

4. **grade_90.png** (90-99 баллов):

- Фон: Светло-зелёный (#66BB6A)
- Текст: "90"
- Сохраните как **grade_90.png**

5. **grade_100.png** (100 баллов):

- Фон: Тёмно-зелёный (#388E3C)
- Текст: "100"
- Добавьте значок ✓ или звезду (можно воспользоваться эмодзи). **Пример:**

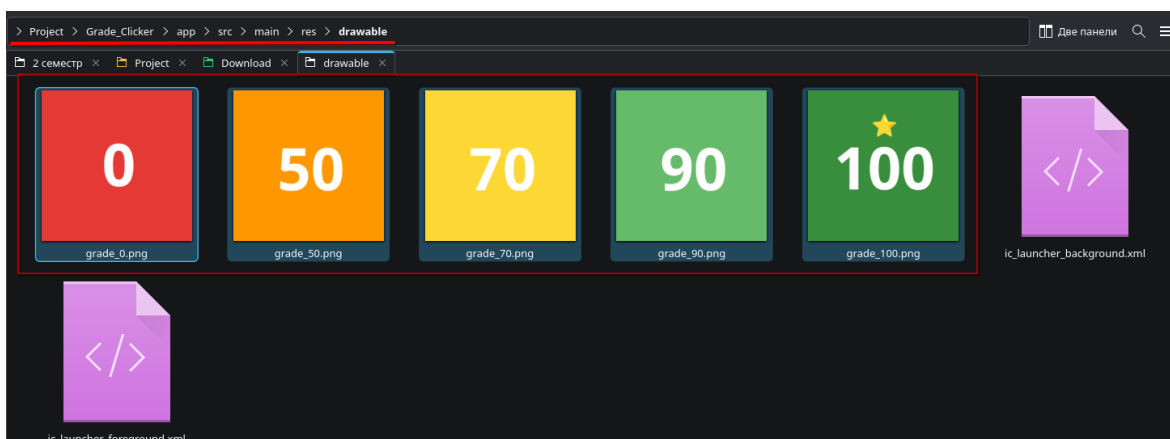
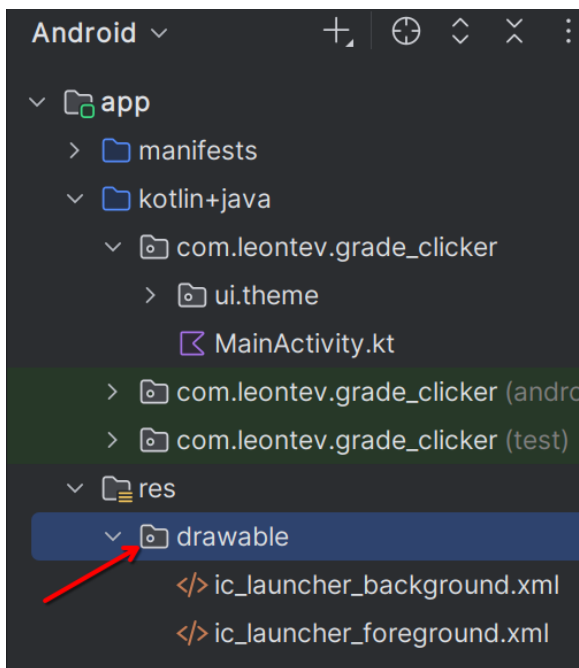


- Сохраните как **grade_100.png**

Альтернатива: Можете найти готовые примеры изображений из интернета.

Добавление изображений в проект

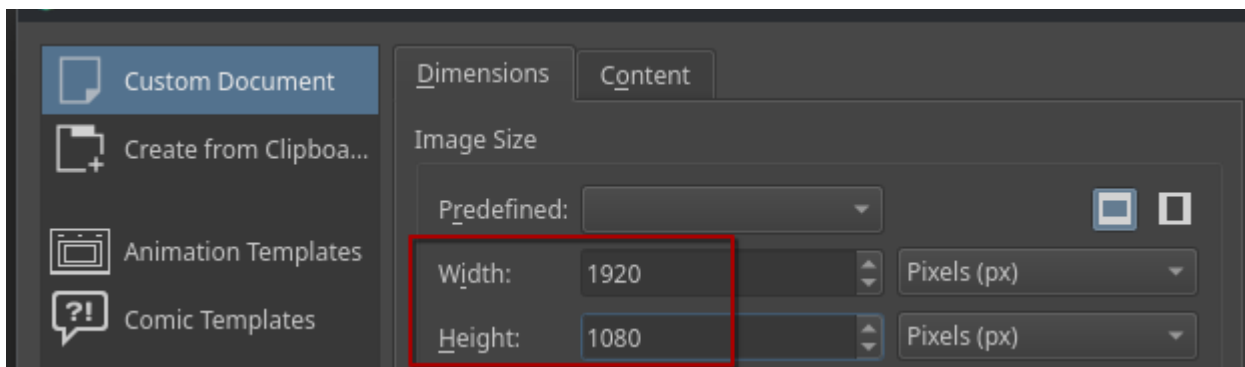
1. Перетащите все 5 файлов в **res/drawable/**



2.2. Создание фонового изображения

Создайте простой фон:

1. В Krita: **File** → **New**
2. Размер: **1920x1080px**

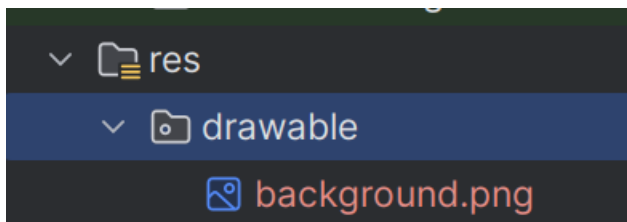


3. Создайте градиент от **синего** к **фиолетовому**. **Пример** реализации:



4. Сохраните как **background.png**

5. Перетащите в **res/drawable/**



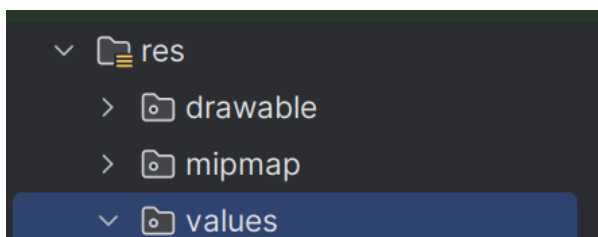
Шаг 3. Настройка ресурсов

3.1. Создание **dimens.xml**

Файл **dimens.xml** используется для хранения размерных значений (отступы, размеры элементов), что позволяет легко изменять их в одном месте для всего приложения.

Создайте файл **res/values/dimens.xml** для хранения размеров:

1. Кликните правой кнопкой на **res/values**



2. **New** → **Values Resource File**

3. Имя файла: **dimens**

4. Нажмите **OK**

Добавьте следующий код:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <dimen name="padding_medium">16dp</dimen>
    <dimen name="image_size">200dp</dimen>
</resources>
```

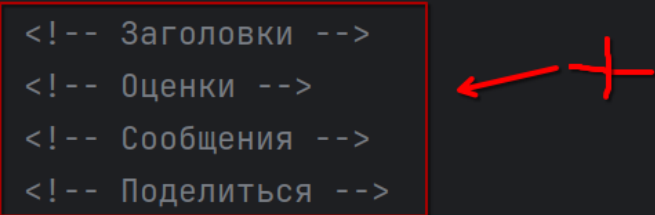
- **padding_medium** — средние отступы
- **image_size** — размер изображений

3.2. Обновление strings.xml

Откройте **res/values/strings.xml**.

Добавьте строки:

```
<resources>
    <string name="app_name">Grade_Clicker</string>
    <!-- Заголовки -->
    <!-- Оценки -->
    <!-- Сообщения -->
    <!-- Поделиться -->
</resources>
```



Заголовки:

```
<string name="points_earned">Набрано баллов</string>
<string name="total_clicks">Всего кликов</string>
<string name="current_grade">Текущая оценка</string>
```

Оценки:

```
<string name="grade_fail">Неудовлетворительно</string>
<string name="grade_pass">Удовлетворительно</string>
<string name="grade_good">Хорошо</string>
<string name="grade_excellent">Отлично</string>
```

Сообщения:

```
<string name="message_fail">Продолжайте работать!</string>
<string name="message_pass">Вы получили зачёт!</string>
<string name="message_good">Отличная работа!</string>
<string name="message_excellent">Вы успешно сдали курс!</string>
```

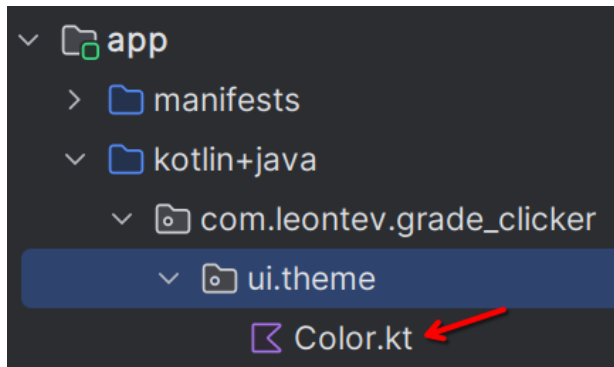
Поделиться:

```
<string name="share">Поделиться</string>
<string name="share_text">Я набрал %1$d баллов по курсу \"Разработка мобильных приложений\"!
```

```
#GradeClicker</string>
<string name="sharing_not_available">Функция недоступна</string>
```

3.3. Обновление Color.kt

Откройте `ui/theme/Color.kt`.



Обновите строки (**не трогайте ваш первый пакет**):

```
тут_будет_ваш_пакет
import androidx.compose.ui.graphics.Color

// Светлая тема
val md_theme_light_primary = Color(0xFF1976D2)
val md_theme_light_onPrimary = Color(0xFFFFFFFF)
val md_theme_light_secondaryContainer = Color(0xFFE3F2FD)
val md_theme_light_onSecondaryContainer = Color(0xFF0D47A1)
val md_theme_light_background = Color(0xFFFFFAFA)

// Тёмная тема
val md_theme_dark_primary = Color(0xFF64B5F6)
val md_theme_dark_onPrimary = Color(0xFF0D47A1)
val md_theme_dark_secondaryContainer = Color(0xFF1565C0)
val md_theme_dark_onSecondaryContainer = Color(0xFFE3F2FD)
val md_theme_dark_background = Color(0xFF121212)
```

Обновите темы (**не трогайте ваш первый пакет**):

```
тут_будет_ваш_пакет

import android.os.Build
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.dynamicDarkColorScheme
import androidx.compose.material3.dynamicLightColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.platform.LocalContext

private val DarkColorScheme = darkColorScheme(
    primary = md_theme_dark_primary,
    onPrimary = md_theme_dark_onPrimary,
    secondaryContainer = md_theme_dark_secondaryContainer,
    onSecondaryContainer = md_theme_dark_onSecondaryContainer,
    background = md_theme_dark_background
)
```

```

private val LightColorScheme = lightColorScheme(
    primary = md_theme_light_primary,
    onPrimary = md_theme_light_onPrimary,
    secondaryContainer = md_theme_light_secondaryContainer,
    onSecondaryContainer = md_theme_light_onSecondaryContainer,
    background = md_theme_light_background
)

@Composable
fun GradeClickerTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Dynamic color is available on Android 12+
    dynamicColor: Boolean = false,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
        }
        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }

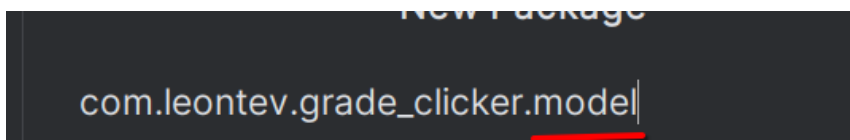
    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}

```

Шаг 4. Создание модели данных

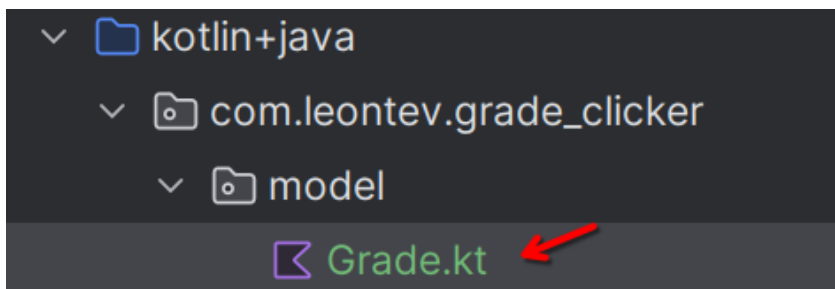
4.1. Создание пакета model

1. Кликните правой кнопкой на **com.ваш_пакет.grade_clicker**
2. **New → Package**
3. Введите: **model**

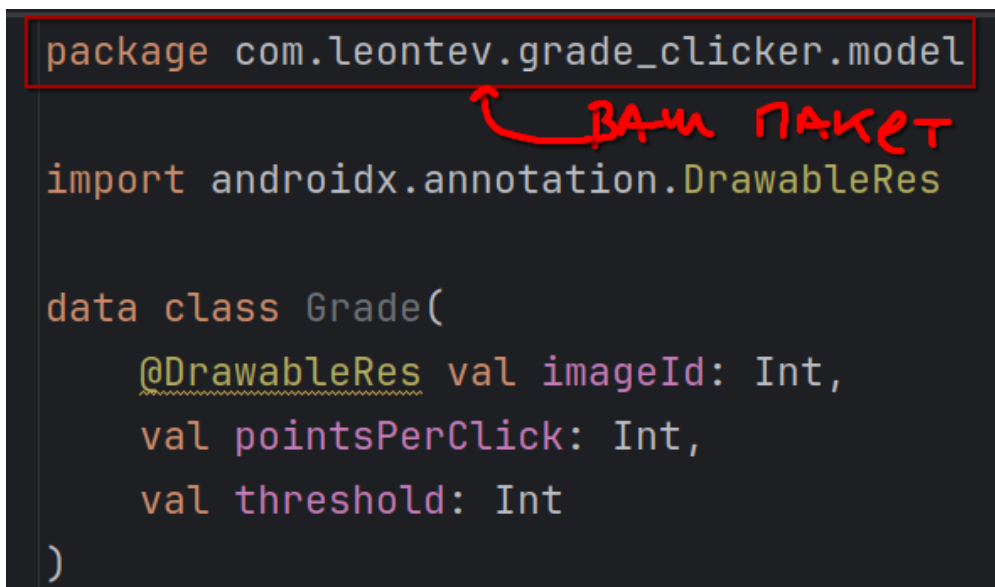


4.2. Создание data class Grade

В пакете **model** создайте файл **Grade.kt**



Добавьте код (**не трогайте ваш первый пакет**):



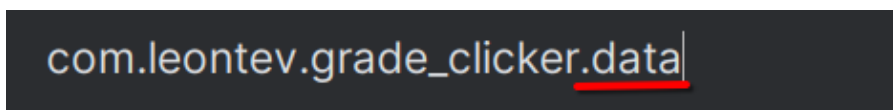
Объяснение:

- **imageId** — ID изображения для этого этапа
- **pointsPerClick** — баллов за клик (всегда 5)
- **threshold** — порог баллов для появления этого изображения

Шаг 5. Создание источника данных

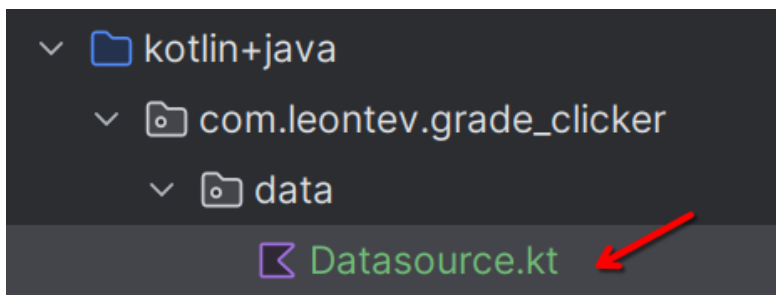
5.1. Создание пакета data

1. Кликните правой кнопкой на **com.ваш_пакет.grade_clicker**
2. **New** → **Package**
3. Введите: **data**



5.2. Создание Datasource.kt

В пакете **data** создайте файл **Datasource.kt**



Добавьте код (**обратите внимание на указание ваших пакетов**):

```
1 package com.leontev.grade_clicker.data
2
3 import com.leontev.grade_clicker.R
4 import com.leontev.grade_clicker.model.Grade
5
6 object Datasource {
7     val gradeList = listOf(
8         Grade(R.drawable.grade_0, 5, 0),
9         Grade(R.drawable.grade_50, 5, 50),
10        Grade(R.drawable.grade_70, 5, 70),
11        Grade(R.drawable.grade_90, 5, 90),
12        Grade(R.drawable.grade_100, 5, 100)
13    )
14 }
```

ВАШИ ПАКЕТЫ

Объяснение:

- Список содержит 5 этапов
- Каждый этап имеет своё изображение и порог баллов
- За каждый клик начисляется 5 баллов

Выполните в терминале:

git add .

git commit -m "Step 5: Basic model and file"

git push

Шаг 6. Создание базового интерфейса

Очистка MainActivity.kt

1. Откройте **MainActivity.kt** и удалите лишние функции, оставив базовую структуру:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            GradeClickerTheme {

            }
        }
    }
}
```

2. Нажмите **Ctrl+Alt+O**, чтобы Android Studio автоматически удалила неиспользуемые импорты.

3. Создание **Composable**-функцию **GradeClickerApp**

Добавьте ниже **MainActivity** функцию:

```
class MainActivity : ComponentActivity() {...}

@Composable
fun GradeClickerApp(grades: List<Grade>) {
}
```

- **@Composable** — функция, которая рисует UI
- **grades** — список уровней/оценок

При необходимости добавляйте импорты:

```
import androidx.compose.runtime.Composable
import com.leontev.grade_clicker.model.Grade
```

4. Хранение состояния приложения

Внутри **GradeClickerApp** вставьте:

```
fun GradeClickerApp(grades: List<Grade>) {
    + [ var points by remember { mutableStateOf(0) }
      var clicks by remember { mutableStateOf(0) }
```

- **points** — общее количество очков
- **clicks** — количество кликов
- **remember** — сохраняет значения

При необходимости добавляйте импорты:

```
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
```

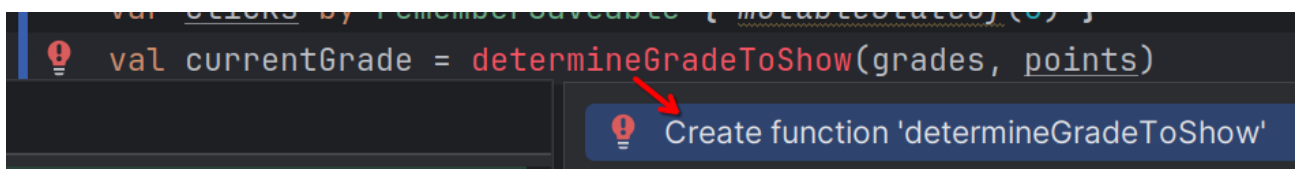
5. Определение текущей оценки

Далее **вставьте**:

```
fun GradeClickerApp(grades: List<Grade>) {
    var points by rememberSaveable { mutableStateOf(0) }
    var clicks by rememberSaveable { mutableStateOf(0) }
    + val currentGrade = determineGradeToShow(grades, points)
```

- Вызывается функция, которая:
 - сравнивает очки
 - определяет, какое изображение и сколько очков за клик использовать

Т.к. у нас нету данной функции, то воспользуемся подсказкой Android Studio (Alt+Enter):



У нас появится новая функция:

```
fun determineGradeToShow(
    grades: List<Grade>,
    points: Int
) {
}
```

В ней мы определим логику определения текущей оценки:

```
fun determineGradeToShow(grades: List<Grade>, points: Int): Grade {
    var gradeToShow = grades.first()
    for (grade in grades) {
        if (points >= grade.threshold) {
            gradeToShow = grade
        } else {
            break
        }
    }
    return gradeToShow
}
```

- Берётся минимальный уровень

- Перебираются оценки по возрастанию
- Если очков хватает — уровень обновляется
- Возвращается последний подходящий уровень

6. Далее внутри **GradeClickerApp** создадим вертикальный контейнер **Column**:

```
Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(dimensionResource(R.dimen.padding_medium)),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.SpaceBetween
) { }
```

- **Column** — вертикальное расположение элементов
- **padding_medium** — отступы из ресурсов
- **SpaceBetween** — элементы равномерно распределены по экрану

При необходимости добавьте импорты:

```
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.dimensionResource
```

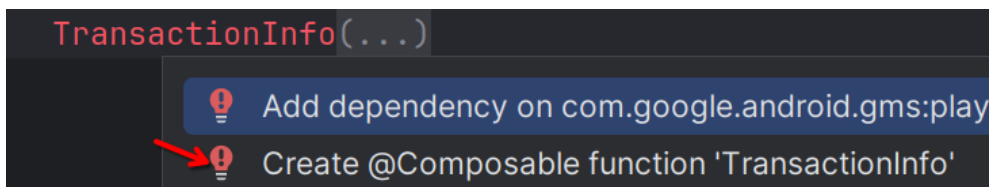
Внутри **Column** вставьте:

```
Column(...)
{
    // Заголовок
    1 Text(...)
    // Изображение (кликабельное)
    2 Image(...)
    // Информация
    3 TransactionInfo(...)
}
```

При необходимости добавьте импорты:

```
import androidx.compose.material3.Text
import androidx.compose.foundation.Image
```

Т.к. у нас нету функции **TransactionInfo()**, то воспользуемся подсказкой Android Studio (**Alt+Enter**):



У нас появится новая функция, наполнением которой мы займёмся позже:

```
@Composable
fun TransactionInfo() {
    TODO("Not yet implemented")
}
```

7. Заголовок приложения

Внутри заголовка **Text** напишите:

```
// Заголовок
Text(
    text = stringResource(R.string.app_name),
    style = MaterialTheme.typography.headlineLarge,
    modifier = Modifier.padding(16.dp)
)
```

- Отображает название приложения
- Берёт текст из strings.xml
- Использует стиль темы

При необходимости добавьте импорты:

```
import androidx.compose.ui.res.stringResource
import androidx.compose.material3.MaterialTheme
import androidx.compose.ui.unit.dp
```

8. Кликабельное изображение

В **Image** вставьте:

```
// Изображение (кликабельное)
Image(
    painter = painterResource(currentGrade.imageId),
    contentDescription = null,
    modifier = Modifier
        .size(dimensionResource(R.dimen.image_size))
        .clickable {
            points += currentGrade.pointsPerClick
            clicks++
        },
    contentScale = ContentScale.Fit
)
```

- **Image** — отображает картинку текущей оценки
- **clickable {}**:
 - увеличивает очки
 - увеличивает счётчик кликов
- **pointsPerClick** зависит от уровня

При необходимости добавьте импорты:

9. Блок с информацией

Вставьте в **TransactionInfo** в функции **GradeClickerApp**:

```
@Composable
fun GradeClickerApp(grades: List<Grade>) {
    ...
    Column(...) {
        // Заголовок
        Text(...)
        // Изображение (кликабельное)
        Image(...)
        // Информация
        TransactionInfo(
            points = points,
            clicks = clicks )
    }
}
```

- Отдельный компонент
- Показывает очки и клики
- Делает код чище и модульнее

Теперь напишем функцию Composable-функцию **TransactionInfo()**

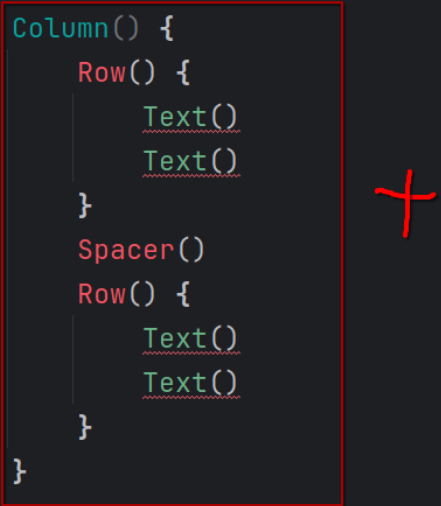
10. Добавьте в **TransactionInfo**:

```
@Composable
fun TransactionInfo(
    points: Int,
    clicks: Int,
    modifier: Modifier = Modifier
) {
    TODO("Not yet implemented")
}
```

- Отвечает только за отображение данных
- Не хранит состояние

Внутри тела функции создайте:

```
@Composable
fun TransactionInfo(
    points: Int,
    clicks: Int,
    modifier: Modifier = Modifier
) {
    Column() {
        Row() {
            Text()
            Text()
        }
        Spacer()
        Row() {
            Text()
            Text()
        }
    }
}
```



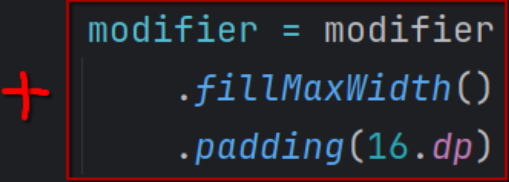
- **Column** — вертикальный контейнер
- **Spacer** — элемент для создания пространства между двумя элементами пользовательского интерфейса
- **Row** — строки:
 - **Очки**
 - **Количество кликов**

При необходимости добавьте импорты:

```
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
```

Внутри **Column** добавьте:

```
Column(
    modifier = modifier
        .fillMaxWidth()
        .padding(16.dp)
) {
    Row(
```



Далее пропишем **первый Row**:

```

Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween
) {
    Text(
        text = stringResource(R.string.points_earned),
        style = MaterialTheme.typography.titleMedium
    )
    Text(
        text = points.toString(),
        style = MaterialTheme.typography.titleMedium
    )
}

```

Отступ:

```
Spacer(modifier = Modifier.height(8.dp))
```

Второй Row:

```

Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween
) {
    Text(
        text = stringResource(R.string.total_clicks),
        style = MaterialTheme.typography.bodyLarge
    )
    Text(
        text = clicks.toString(),
        style = MaterialTheme.typography.bodyLarge
    )
}

```

При необходимости добавьте импорты:

```

import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height

```

11. Создадим функцию **Preview** для предпросмотра:

```

@Preview
@Composable
fun GradeClickerPreview() {
    GradeClickerTheme {
        GradeClickerApp(grades = Datasource.gradeList)
    }
}

```

- Показывает экран в Android Studio
- Не запускает приложение на устройстве
- Удобно для быстрой проверки UI

При необходимости добавьте импорты:

```
import androidx.compose.ui.tooling.preview.Preview
import com.leontev.grade_clicker.data.Datasource
```

В **MainActivity** внутри **GradeClickerTheme** запустите основной экран приложения:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            GradeClickerTheme {
                Surface(
                    modifier = Modifier.fillMaxSize()
                ) {
                    GradeClickerApp(grades = Datasource.gradeList)
                }
            }
        }
    }
}
```

- Запускается **Composable-функция** GradeClickerApp
- В неё передаётся список оценок (gradeList)
- Datasource — источник данных (модель приложения)

При необходимости добавьте импорты:

```
import androidx.compose.material3.Surface
```

Шаг 7. Запуск приложения

1. Запустите приложение на эмуляторе и в предварительном просмотре в интерактивном режиме.
2. Проверьте работу:
 - Изображение меняется при достижении порогов (50, 70, 90, 100 баллов)
 - Счётчики обновляются
 - Каждый клик добавляет 5 баллов

Скриншоты для отчёта

Сделайте **пять** скриншотов в папку **app/img** разных этапов:

1. 0 баллов: **step7_grade_0_FIO.png**
2. 50 баллов: **step7_grade_50_FIO.png**
3. 70 баллов: **step7_grade_70_FIO.png**
4. 90 баллов: **step7_grade_90_FIO.png**
5. 100 баллов: **step7_grade_100_FIO.png**

Выполните команды в терминале:

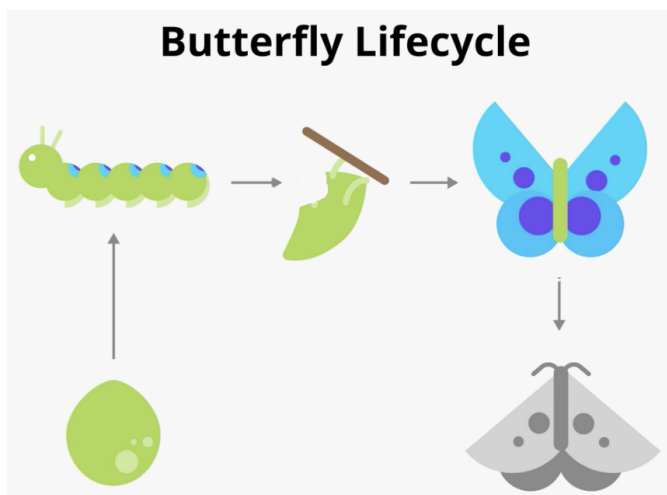
git add .

git commit -m "Step 7: Basic Grade Clicker app completed"

git push

Шаг 8. Изучение методов жизненного цикла и добавление логирования

Каждая **Activity** имеет так называемый **жизненный цикл (lifecycle)**. Этот термин является аллюзией на жизненные циклы растений и животных, как жизненный цикл бабочки — различные состояния бабочки показывают её рост от яйца к гусенице, к куколке, к бабочке и к смерти.



Аналогично, жизненный цикл **Activity** состоит из различных **состояний**, через которые проходит **Activity** от момента инициализации до её уничтожения, когда операционная система освобождает её память.

Теоретическая часть. Что такое Activity Lifecycle?

Activity Lifecycle (Жизненный цикл Activity) — это последовательность состояний, через которые проходит Activity от момента создания до уничтожения.

В течение своей жизни Activity переходит между различными **состояниями**. Эти переходы называются **жизненным циклом Activity**.

Зачем нужно понимать жизненный цикл?

Если ваши Activity **неправильно реагируют** на изменения состояния, приложение может:

- Генерировать странные баги
- Запутывать пользователей непонятным поведением
- Использовать слишком много системных ресурсов
- Терять данные пользователя

Понимание и правильная обработка lifecycle — важная часть Android-разработки!

Основные состояния Activity

Activity может находиться в одном из следующих состояний:



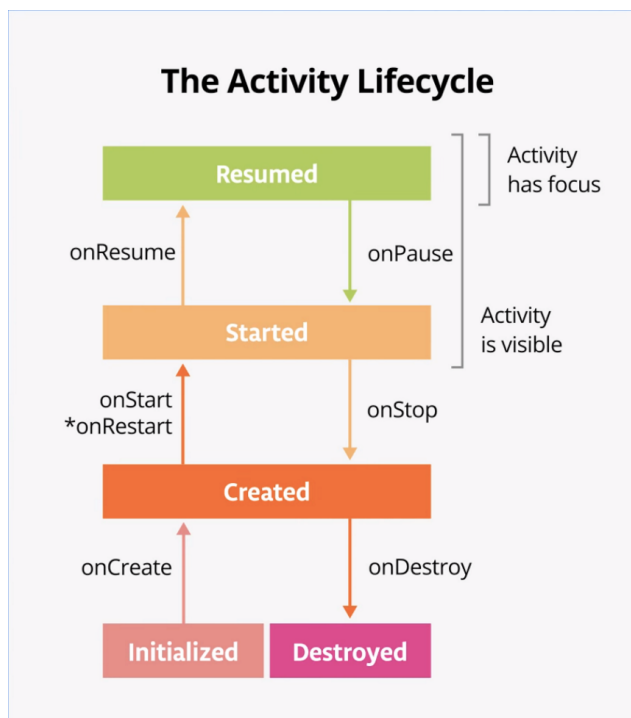
Состояние	Описание
Created	Activity создана, но ещё не видна
Started	Activity видна, но не в фокусе
Resumed	Activity на переднем плане и активна
Paused	Activity частично видна (например, диалог поверх)
Stopped	Activity не видна (пользователь перешёл в другое приложение)
Destroyed	Activity уничтожена, ресурсы освобождены

Callback-методы жизненного цикла

Android вызывает специальные методы при переходе между состояниями:

- **onCreate()** - Activity создаётся

- **onStart()** - Activity становится видимой
- **onResume()** - Activity получает фокус (активна)
- **onPause()** - Activity теряет фокус
- **onStop()** - Activity становится невидимой
- **onDestroy()** - Activity уничтожается
- **onRestart()** - Activity перезапускается после Stop



8.1. Изучение метода onCreate()

Метод `onCreate()` — это единственный метод, который должна реализовывать каждая Activity.

У нас уже определён метод `onCreate()`:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContent {
        GradeClickerTheme {
            Surface(
                modifier = Modifier.fillMaxSize()
            ) {
                GradeClickerApp(grades = Datasource.gradeList)
            }
        }
    }
}
```

Что происходит в onCreate():

- Вызывается **один раз** при создании Activity
- Здесь выполняется **инициализация**: установка UI через setContent()
- Необходимо вызвать super.onCreate() для завершения создания Activity

8.2. Добавление логирования

Логирование позволяет записывать короткие сообщения в консоль и отслеживать, когда вызываются различные callback-методы.

Создание константы TAG

В самом начале файла MainActivity.kt, перед объявлением класса, добавьте:

```
private const val TAG = "MainActivity" +  
1 Usage  
class MainActivity : ComponentActivity() {
```

- const val TAG — константа времени компиляции
- "MainActivity" — тег для фильтрации логов в Logcat
- private — доступна только в этом файле

Добавление лога в onCreate()

В методе onCreate(), после вызова super.onCreate(), добавьте:

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        + Log.d(TAG, "onCreate Called")  
        enableEdgeToEdge()  
    }  
}
```

Объяснение Log.d():

- Log.d() — записывает **debug-сообщение** в Logcat
- Первый параметр — **тег** (TAG)
- Второй параметр — **сообщение** ("onCreate Called")

Необходимый импорт:

```
import android.util.Log
```

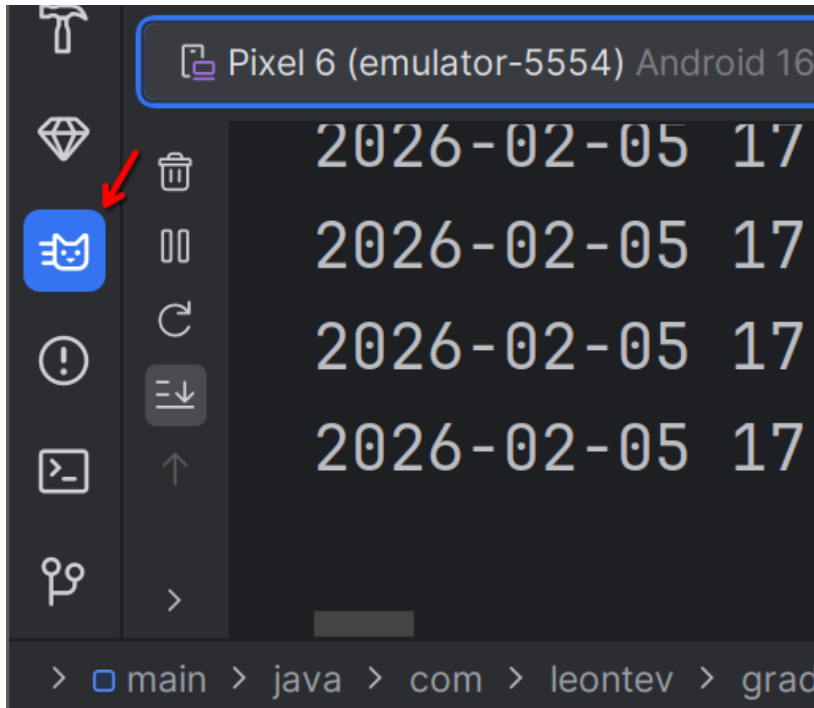
Другие методы Log:

- Log.v() — verbose (подробные сообщения)

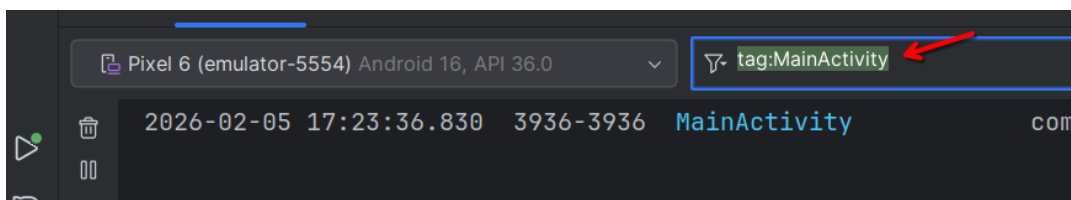
- `Log.i()` — информационные сообщения
- `Log.w()` — предупреждения
- `Log.e()` — ошибки

8.3. Просмотр логов в Logcat

1. Запустите приложение (Run)
2. В Android Studio внизу найдите вкладку **Logcat**



3. В поле поиска **Logcat** введите: `tag:MainActivity`



4. Вы увидите приблизительное **сообщение**:

2026-02-05 17:23:36.830 3936-3936 MainActivity com.leontev.grade_clicker D onCreate Called

Что означает каждая часть:

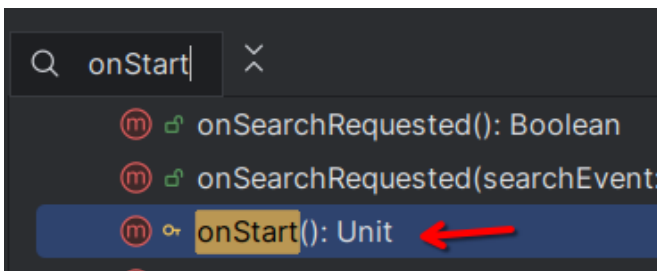
- 2026-02-05 17:23:36.830 — дата и время
- 3936-3936 — ID процесса и потока
- MainActivity — тег (TAG)
- com.leontev.grade_clicker — имя пакета
- D — уровень Debug
- onCreate Called — наше сообщение

8.4. Добавление метода onStart()

Теперь переопределим метод `onStart()` и добавим логирование.

В классе MainActivity:

1. Поставьте курсор **внутри** класса MainActivity
2. Нажмите **Code** → **Override Methods...** (или **Ctrl+O**)
3. Начните вводить `onStart`
4. Выберите `onStart()` из списка → **ОК** (можно переключаться стрелками между методами)



Android Studio вставит код:

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {...}  
    override fun onStart() {  
        super.onStart()  
    }  
}
```

A screenshot of the Android Studio code editor showing the implementation of the `onStart()` method in the `MainActivity` class. The code is as follows: `class MainActivity : ComponentActivity() {`, `override fun onCreate(savedInstanceState: Bundle?) {...}`, `override fun onStart() {`, `super.onStart()`, `}`, `}`. A red rectangle highlights the `onStart()` method block, and a red arrow points to it from the right.

Добавьте логирование:

```
override fun onStart() {  
    super.onStart()  
    Log.d(TAG, "onStart Called")  
}
```

A screenshot of the Android Studio code editor showing the `onStart()` method with logging added. The code is: `override fun onStart() {`, `super.onStart()`, `Log.d(TAG, "onStart Called")`, `}`. A red plus sign is placed to the left of the `Log.d` line, indicating it has been added.

8.5. Добавление остальных lifecycle-методов

Переопределите **все остальные** методы жизненного цикла:

Добавьте в класс MainActivity:

```
override fun onResume() {  
    super.onResume()  
    Log.d(TAG, "onResume Called")  
}
```

```
override fun onRestart() {  
    super.onRestart()  
    Log.d(TAG, "onRestart Called")  
}
```

```
override fun onPause() {  
    super.onPause()  
    Log.d(TAG, "onPause Called")  
}
```

```
override fun onStop() {  
    super.onStop()  
    Log.d(TAG, "onStop Called")  
}
```

```
override fun onDestroy() {  
    super.onDestroy()  
    Log.d(TAG, "onDestroy Called")  
}
```

8.6. Тестирование жизненного цикла

Теперь протестируем различные сценарии использования приложения.

Сценарий 1: Запуск приложения

1. Запустите приложение
2. Откройте Logcat, фильтр: tag:MainActivity
3. Вы увидите:

```
D onCreate Called  
D onStart Called  
D onResume Called
```

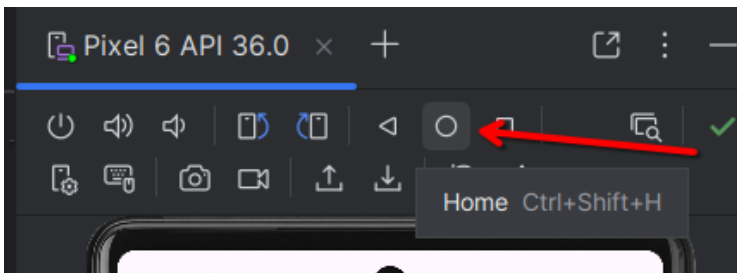
Объяснение:

- При запуске вызываются три метода по порядку

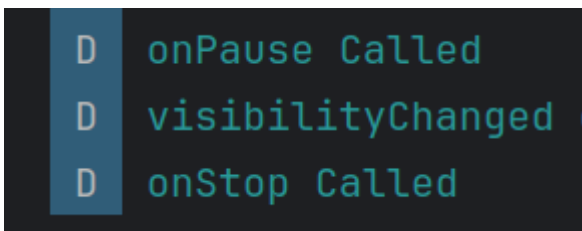
- `onCreate()` — создание Activity
- `onStart()` — Activity становится видимой
- `onResume()` — Activity получает фокус (активна)

Сценарий 2: Нажатие кнопки Home

1. Приложение запущено
2. **Наберите несколько баллов** (кликните несколько раз)
3. **Нажмите кнопку Home** на устройстве



4. Проверьте **Logcat**:

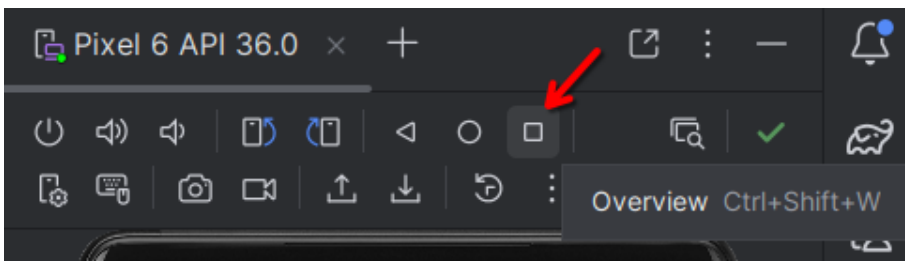


Что произошло:

- Приложение **ушло в фон** (background)
- `onPause()` — потеряли фокус
- `onStop()` — стали невидимы
- Activity **НЕ уничтожена**, остаётся в памяти

Возврат из Recents

1. Откройте **Overview** (кнопка многозадачности □)



2. **Верните приложение** на передний план
3. Проверьте **Logcat**:

```
D onRestart Called
D onStart Called
D onResume Called
```

Что произошло:

- `onRestart()` — вызывается только при **перезапуске** после `Stop`
- `onStart()` — снова стали видимыми
- `onResume()` — получили фокус

Важно: `onCreate()` **НЕ** вызывается! Activity не создавалась заново, она была в памяти.

Сценарий 3: Переключение между приложениями (работало на старых версиях)

1. Приложение запущено.
2. Свайпните **вверх** от нижнего края
3. **НЕ выбирайте** другое приложение, просто посмотрите на список
4. Проверьте Logcat:

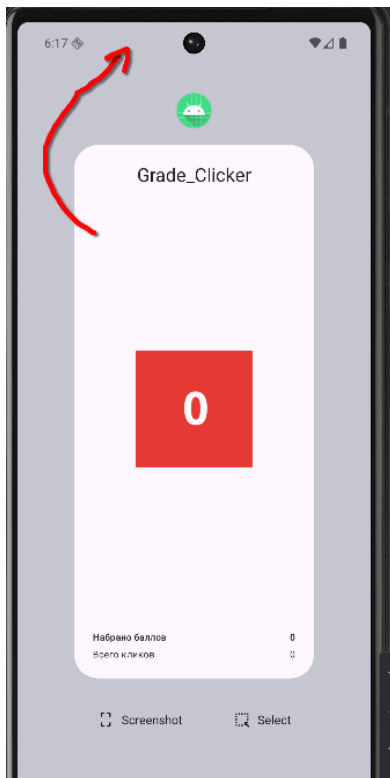
```
D onPause Called
D visibilityChanged ol
D onStop Called
```

- На **старых версиях** Android: при открытии Recent apps вызывался только `onPause()`
- На **новых версиях** (Android 10+): вызываются `onPause()` и `onStop()`, так как приложение полностью уходит в фон
- Activity **невидима**, но **НЕ уничтожена** (остаётся в памяти)

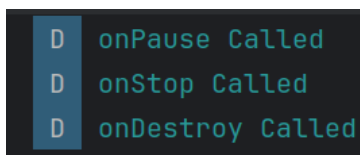
Примечание: Современные версии Android агрессивнее управляют lifecycle, поэтому различие между `onPause()` и `onStop()` стало менее очевидным в повседневном использовании.

Сценарий 4: Заккрытие приложения

1. Приложение запущено
2. Свайпните **вверх** от нижнего края
3. Удалите приложение из списка переносом **вверх**:



4. Проверьте **Logcat**:



Что должно произойти:

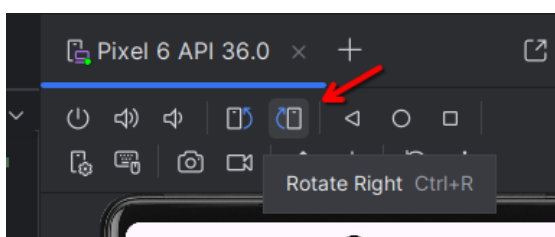
- Activity **полностью уничтожена**
- `onDestroy()` освобождает ресурсы
- При следующем запуске вызовется `onCreate()` заново

Шаг 9. Проблема: Потеря данных при повороте экрана

Сейчас мы обнаружим **критическую проблему** нашего приложения.

9.1. Воспроизведение проблемы

1. Запустите приложение
2. Наберите **50+ баллов** (кликните ~10 раз)
3. Убедитесь, что изображение изменилось (должен показываться `grade_50`)
4. Поверните устройство



5. Проблема: Баллы сбросились на 0!

9.2. Анализ проблемы в Logcat

Проверьте Logcat при повороте:

```
D onPause Called
D onStop Called
D onDestroy Called
D onCreate Called
D onStart Called
D onResume Called
```

Что произошло:

- Поворот экрана вызывает **configuration change** (изменение конфигурации)
- Android полностью уничтожает Activity (`onDestroy`)
- Затем создаёт заново (`onCreate`)
- Все данные **теряются!**

Шаг 10. Решение: Использование `rememberSaveable`

Чтобы **сохранить** данные при configuration changes, используем `rememberSaveable`.

Теория: `remember` vs `rememberSaveable`

API	Сохраняет при	НЕ сохраняет при
<code>remember</code>	Рекомпозиции	Configuration changes
<code>rememberSaveable</code>	Рекомпозиции	Configuration changes

`remember`:

- Сохраняет значение **только** при рекомпозиции
- Теряет значение при **пересоздании Activity**

`rememberSaveable`:

- Сохраняет значение при **рекомпозиции**
- **И ПРИ** configuration changes (поворот, смена языка и т.д.)

10.1. Обновление кода

Откройте `MainActivity.kt`, найдите функцию `GradeClickerApp()`.

Обновите:

```
fun GradeClickerApp(grades: List<Grade>) {
    var points by rememberSaveable { mutableStateOf(0) }
    var clicks by rememberSaveable { mutableStateOf(0) }
    val currentGrade = determineGradeToShow(grades, points)
```

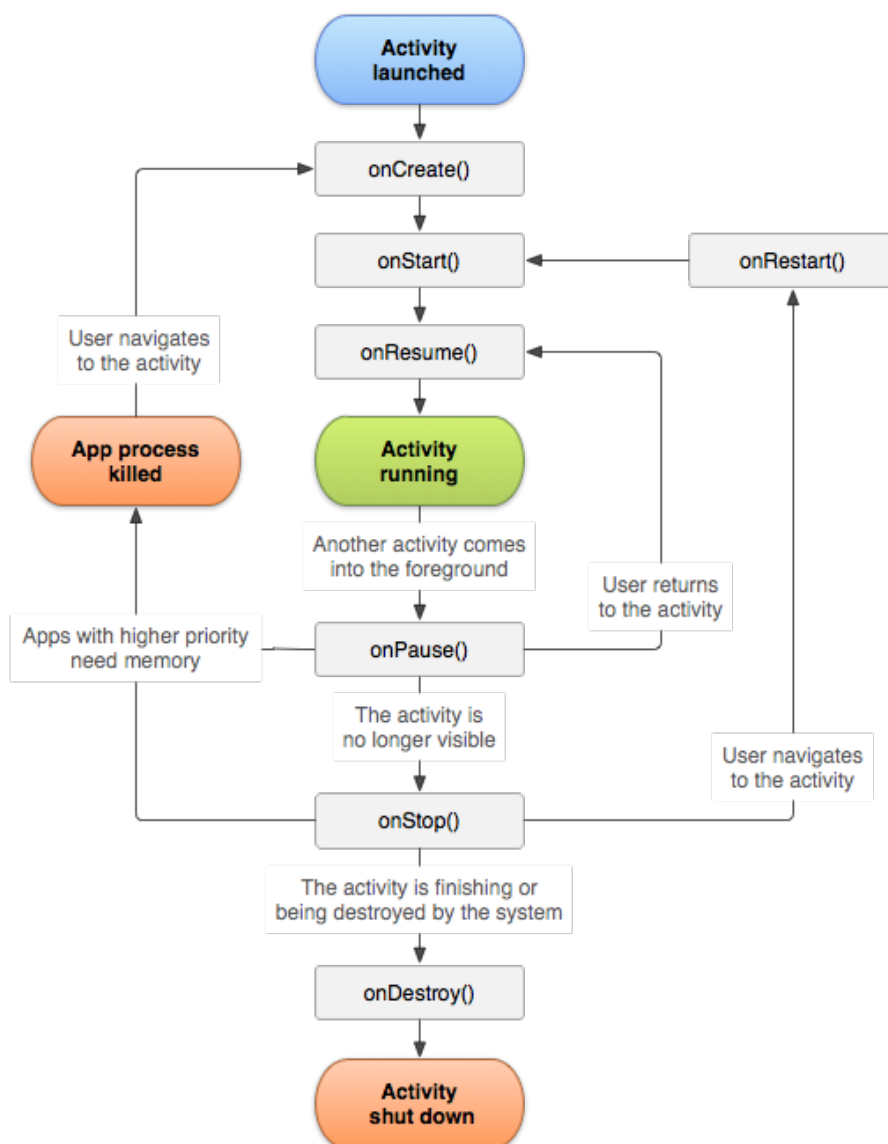
Необходимый импорт:

10.2. Тестирование решения

1. Запустите приложение
2. Наберите 70+ баллов
3. Поверните устройство
4. Результат: Баллы сохранились!

Проверьте Logcat: Activity пересоздалась, но данные сохранились благодаря rememberSaveable!

Диаграмма жизненного цикла



Скриншоты для отчёта

Сделайте **четыре** скриншота Logcat:

1. Запуск приложения:

- Запустите приложение

- Скриншот Logcat с onCreate, onStart, onResume
- Сохраните как: **step10_app_start_logcat_FIO.png**

2. Home и возврат:

- Нажмите Home → вернитесь
- Скриншот Logcat с onPause, onStop, onRestart, onStart, onResume
- Сохраните как: **step10_home_return_logcat_FIO.png**

3. Поворот (без rememberSaveable):

- Временно замените rememberSaveable на remember
- Наберите баллы, поверните экран
- Скриншот с потерей данных
- Сохраните как: **step10_rotation_data_loss_FIO.png**

4. Поворот (с rememberSaveable):

- Верните rememberSaveable
- Наберите баллы, поверните экран
- Скриншот с сохранением данных
- Сохраните как: **step10_rotation_data_saved_FIO.png**

git add .

git commit -m "Step 10: Lifecycle callbacks and rememberSaveable implemented"

git push

Самостоятельное задание: Оформление README.md

Создайте файл **README.md** в корне проекта, который будет описывать вашу лабораторную работу.

Требования к README.md:

Файл должен содержать следующие разделы (напишите их самостоятельно):

1. **Название проекта** — название и краткое описание приложения
2. **Цель работы** — что вы изучали в этой лабораторной
3. **Ответьте на контрольные вопросы:**
 - Какие три метода вызываются при запуске приложения?
 - Чем отличается onPause() от onStop()?
 - Когда вызывается onRestart()?
 - Что происходит при повороте экрана?

- В чём разница между `remember` и `rememberSaveable`?
- Почему `onCreate()` должен вызывать `super.onCreate()`?
- Для чего используется класс `Log`?
- Какой метод вызывается только один раз за время жизни `Activity`?

4. **Автор** — ваше ФИО и группа

После создания **README.md**:

git add README.md

git commit -m "Add README documentation"

git push